

顾客类重构方案：模板与实例分离

日期：2026-07-06

状态：设计阶段

优先级：中低（不阻塞当前开发）

一、现状

当前系统中存在两套顾客数据，但共用同一个 `Customer` 类（824行，40+字段）：

容器	所属	含义	加载来源
<code>CustomerManager::Customers</code>	<code>GameScene</code>	模板/蓝图	<code>save/Customers.rui</code>
<code>GameWorld::customers</code>	<code>GameWorld</code>	活跃实例	从模板 Init 产生

两者的关系是：模板是"这个顾客叫什么、喜欢什么"，实例是"这个顾客此刻在店内干什么"。但由于用的是同一个类，字段全部混在一起，每次生成顾客都要整对象拷贝，修改偏好时又要双向同步。

二、问题分析

2.1 双向同步

```
// Customer::Update 把模板引用传进去
customers[i].Update(chairs, ..., customerManager.Customers[j], ...);

// Customer::Eat() 内部：
customer.AddPreference(...); // ① 直接改模板
this->preference = customer.GetCustomerPreference(); // ② 再拷回实例
```

模板是蓝图数据，却在运行时被反复修改。谁改了模板、什么时候改的，调用链隐式且难以追踪。

2.2 整体拷贝

```
// GameScene::onUpdate() - 只为了让 CheckEvent 检查解锁条件
std::vector<Customer> customers = world.GetCustomers(); // 整 vector 深拷贝
checkEvent.update(..., customers);
world.SetCustomers(customers); // 再拷回去
```

每帧一次完整 `vector<Customer>` 拷贝。Customer 有 40+ 字段，顾客多时开销可观。

2.3 hasJoined 语义模糊

```
class Customer {
    bool hasJoined; // 是"模板是否已生成"还是"实例是否已入座"?
};
```

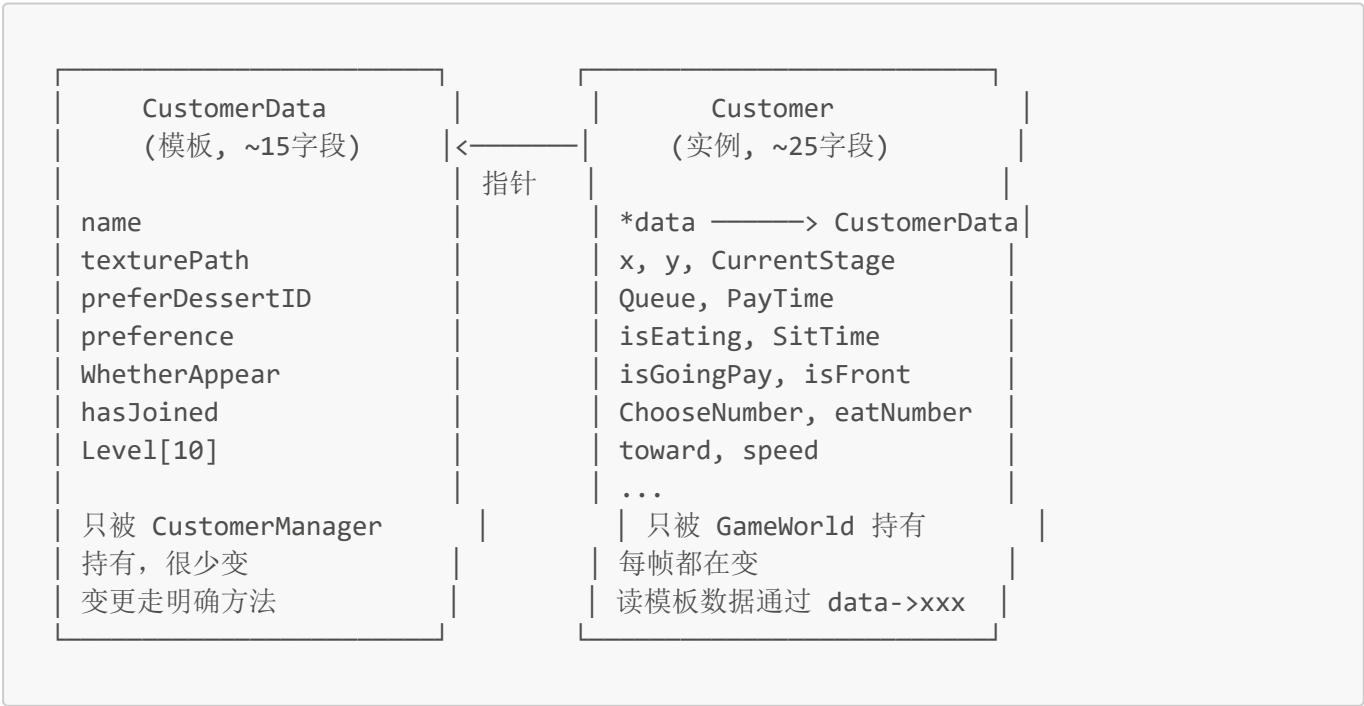
hasJoined 实际标记的是"这个模板对应的顾客当前是否在店内"，防止重复生成。但它定义在 Customer 类里，活跃实例也能无意义地访问它。

2.4 字段混居

Customer 的 ~40 个字段中，哪些属于模板、哪些属于实例，没有任何隔离：

模板字段（长期/少变）	实例字段（每帧变化）
CustomerName	x, y
PathName	CurrentStage
PreferDessertID	Queue
preference	PayTime
WhetherAppear	isEating, SitTime
hasJoined	isGoingPay, isFront
Level[10]	ChooseNumber, eatNumber
	toward, speed
	ChooseTime, EatTime
	isHovered, onSeat
	payCharm, CakeTexture...

三、方案：拆分为 CustomerData + Customer



3.1 CustomerData（新建）

```

struct CustomerData
{
    int id;
    std::string name;
    std::string texturePath;
    int preferDessertID;
    int preference; // 好感度，会随用餐增加
    bool hasAppeared; // 是否已解锁
    bool isInStore; // 当前是否在店内（原 hasJoined）
    int levels[10]; // 好感等级阈值

    // 偏好值变更（唯一修改入口）
    void AddPreference(int dessertID, int count);
    int GetAddNumber(int dessertID, int count) const;
    bool CanSpawn(int currentCustomerCount, const Clock& clock) const;
};

```

- 只被 `CustomerManager` 持有 (`std::vector<CustomerData>`)
- 从 `save/Customers.rui` 加载，修改后在 `onExit` 保存
- `AddPreference` 是唯一修改偏好值的入口，调用链清晰

3.2 Customer（改造）

```

class Customer
{
public:
    // 从模板初始化（只复制一次）
    void InitFromData(const CustomerData* data, int instanceID);

    // 读模板数据（通过指针，零拷贝）
    const std::string& GetName() const { return data->name; }
    int GetPreference() const { return data->preference; }
    // ...

    // 运行时状态
    void Update(/* ... */);
    void OnRender(SDL_Renderer* r);

private:
    const CustomerData* data = nullptr; // ★ 指向模板，只读
    int instanceID;

    // 以下都是运行时状态
    int x, y;
    CustomerStage stage;
    int queuePos, payTime;
    // ...（原先的实例字段）
};

```

3.3 交互流程

```
// 生成顾客 (CustomerManager)
CustomerData& data = GetData(i);
if (data.CanSpawn(world.GetCustomerCount(), clock))
{
    Customer c;
    c.InitFromData(&data, nextID++);
    data.isInStore = true;
    world.AddCustomer(c);
}

// 顾客离店 (Customer::Eat 改为操作 data 指针)
data->AddPreference(dessertID, chooseNumber);
// preference 直接写在 data 上, 实例通过 data->preference 自动可见
// 不再需要 "拷回实例" 这一步

// CheckEvent 更新
checkEvent.update(customerManager, ..., world.GetCustomers());
// world.GetCustomers() 返回 const ref, CheckEvent 只读不写
// 不再需要 copy-out / copy-back

// 离店清理
data->isInStore = false;
world.DeleteCustomer(id);
```

四、同步模式对比

	改造前	改造后
数据流向	模板 ↔ 实例（双向拷贝）	模板 ← 实例（单向写入）
实例 → 模板	手动拷 preference + hasJoined	data->AddPreference()
模板 → 实例	InitCustomer 全量拷贝	data 指针，零拷贝
CheckEvent 交互	GetCustomers() 深拷贝 → SetCustomers() 写回	返回 const ref，只读
hasJoined 归属	在 Customer 上，语义模糊	在 CustomerData 上，语义清晰
内存	模板 N × 40字段 + 实例 N × 40字段	模板 N × 15字段 + 实例 N × 25字段 + N 个指针

五、实施要点

5.1 影响范围

文件	改动程度
include/RUI_Customer.h	大改 — 拆出 CustomerData，Customer 精简
include/RUI_CustomerManager.h	中改 — 容器类型改为 CustomerData
include/RUI_GameWorld.h	小改 — 接口加 const
src/RUI_GameWorld.cpp	小改 — spawn 逻辑调整
src/RUI_GameScene.cpp	小改 — 删除 copy-out/copy-back
include/RUI_CheckUpdate.h	小改 — update 参数调整
include/RUI_GameSerializer.h	小改 — Save 读 data 字段

5.2 风险点

- Customer::Update 内部对模板的修改 (AddPreference) 要全部改为 data->AddPreference
- CheckEvent::UnlockCustomer 中修改 hasJoined 的逻辑要转移到 CustomerData::isInStore
- 存档格式可不改 (CustomerData 的字段序列化后与原先一致)
- 旧存档兼容性：加载时按旧格式读入，填充到 CustomerData

5.3 验证标准

- ☐ 顾客生成/离开后，重新进入场景，偏好值正确保留
- ☐ CheckEvent 解锁逻辑正常工作
- ☐ 存档 → 退出 → 读档，顾客状态一致
- ☐ 编译零警告
- ☐ 帧率无明显下降（实际应因减少拷贝而提升）

六、与其他重构的关系

重构项	状态	依赖
GameEvent → GameWorld	✅ 已完成	无
家具系统	📄 设计中	无
顾客模板分离	📄 本文档	无（可独立进行）
CompetitionScene	➡️ SOON 开发中	无

顾客模板分离与其他重构没有依赖关系，可以在任意时间点独立进行。